Contents

01-goja-schema-map-protobuf-namespace-analysis-and-implementation-guide

Goja schema map protobuf namespace analysis and implementation guide

Executive summary

The `sessionstream` JavaScript module advertises a bulk schema API where callers can write:

```
const ss = require("sessionstream");
const pb = require("sessionstream.examples.chatdemo.v1");

const schemas = ss.schemas({
```

```
  commands: {
    ChatStartInference: pb.StartInferenceCommand,
  },
});
```

That API is not currently reliable. The current implementation decodes the entire JavaScript object through `goja.Runtime.ExportTo` into a Go struct containing `map[string]any` fields. This creates two distinct bugs:

1. The public TypeScript contract uses lower-case section keys (`commands`, `events`, `uiEvents`, `entities`), but the observed `ExportTo` behavior did not populate the Go struct fields from those lower-case keys in the reproduction script. The resulting registry is empty, and later `hub.submit(...)` fails with `unknown command "ChatStartInference"`.
2. If the caller accidentally uses capitalized section names (`Commands`) so that `ExportTo` does populate the Go struct, each generated protobuf namespace object is reduced to a plain `map[string]any`. That strips the hidden `protogoja` schema/prototype token. The exported map keeps public properties such as `typeName`, `builder`, `from`, `is`, and `clone`, but `api_schemas.go` currently only recognizes a public `type` property in map specs.

The proper local fix is to stop using `ExportTo` for schema leaves. `ss.schemas(input)` should walk the original `goja.Value` object, read the four known sections directly, iterate each section object's own keys, and pass each original leaf `goja.Value` to the same resolver used by the fluent `schemas.registerCommand(...)` methods. That preserves hidden generated-protobuf metadata and aligns the bulk API with the fluent API.

A small `typeName` fallback is still useful for plain object descriptors and for compatibility with any code that serializes schema descriptors intentionally. However, the fallback is not a substitute for preserving the original `goja.Value`: `typeName` is public and spoofable, while `protogoja.MessagePrototypeFromValue` recovers a Go-owned hidden token.

Changing `go-go-goja` itself is optional, not required for the sessionstream fix. The best upstream improvement would be documentation and perhaps a small helper for public `typeName` extraction, not a change that makes `MessagePrototypeFromValue` trust public object properties. The hidden token should remain the authoritative proof that a value is a generated namespace object.

Problem statement and scope

User-visible problem

The advertised API says this should work:

```
export interface SchemaMap {
  commands?: Record<string, string | MessageNamespace>;
  events?: Record<string, string | MessageNamespace>;
  uiEvents?: Record<string, string | MessageNamespace>;
  entities?: Record<string, string | MessageNamespace>;
}

export function schemas(input?: SchemaMap): Schemas;
```

Source: `pkg/js/modules/sessionstream/typescript.go:8-24`.

In JavaScript, `MessageNamespace` means values such as `pb.StartInferenceCommand`. These are generated by `protoc-gen-goja-builder` and represent protobuf message types. They are schema tokens, not payloads.

Implementation scope

This guide covers:

- `sessionstream/pkg/js/modules/sessionstream/api_schemas.go`, where bulk schema input is decoded and schema names are registered.
- `sessionstream/pkg/js/modules/sessionstream/module.go`, where protobuf full names are resolved and generated namespace hidden tokens are consumed.
- `sessionstream/pkg/js/modules/sessionstream/codec.go`, where payload values are decoded separately from schema tokens.
- `go-go-goja/pkg/protogoja`, which defines hidden references for generated protobuf namespace objects and built protobuf message objects.
- `go-go-goja/cmd/protoc-gen-goja-builder`, which generates the JavaScript namespace API and TypeScript declarations.

This guide intentionally does not implement code. It records the current evidence, design choice, and implementation plan for a follow-up patch.

Terms and mental model

Schema registry

`sessionstream.SchemaRegistry` stores prototype protobuf messages keyed by logical names such as `ChatStartInference` or `ChatMessage`. It clones registered messages on input and lookup.

Relevant implementation:

- `pkg/sessionstream/schema.go:11-18`: registry maps for commands, events, UI events, and timeline entities.
- `pkg/sessionstream/schema.go:29-43`: public registration methods.
- `pkg/sessionstream/schema.go:85-101`: `register` validates names and clones prototypes.
- `pkg/sessionstream/schema.go:104-127`: lookup and instantiation clone or create fresh messages.

Generated protobuf namespace object

A generated namespace object is the JavaScript value exported as `pb.StartInferenceCommand`. It represents a protobuf type. It is not a concrete protobuf message payload.

Generated TypeScript shape:

```
export interface MessageNamespace<TMessage, TBuilder> {
  readonly typeName: string;
  builder(): TBuilder;
  from(value: TMessage): TMessage;
  is(value: unknown): value is TMessage;
  clone(value: TMessage): TMessage;
}
```

Evidence:

- `examples/chatdemo/gen/sessionstream/examples/chatdemo/v1/chat_goja.pb.go:45-57`: generated TypeScript namespace interface.
- `examples/chatdemo/gen/sessionstream/examples/chatdemo/v1/chat_goja.pb.go:370-390`: `NewStartInferenceCommandGojaNamespace` constructs a namespace object, attaches a hidden prototype, and sets public `typeName` and `builder`.

- `go-go-goja/cmd/protoc-gen-goja-builder/internal/generator/generator.go:420-475`: generator template for namespace construction.

Built protobuf message value

A built message is a JavaScript value returned by:

```
pb.StartInferenceCommand.builder().prompt("hello").build()
```

This value carries a hidden `protogoja.MessageRef`. Go can recover the concrete protobuf payload with `protogoja.MessageFromValue(value)`.

Evidence:

- `go-go-goja/pkg/protogoja/ref.go:57-99`: `ToValue` wraps a protobuf message in a JavaScript object with `typeName`, `toJSON`, `clone`, and `equals`.
- `go-go-goja/pkg/protogoja/ref.go:102-128`: `MessageFromValue` and `MessageRefFromValue` recover the hidden message reference.
- `go-go-goja/cmd/protoc-gen-goja-builder/internal/generator/generator.go:497-503`: generated `build()` returns `protogoja.ToValue(vm, builder.Build())`.

Hidden prototype token

A generated namespace object carries a hidden Go-owned prototype token. Go can recover it with `protogoja.MessagePrototypeFromValue(value)`. This is the correct schema-token path because it does not trust public JavaScript properties.

Evidence:

- `go-go-goja/pkg/protogoja/prototype.go:11`: hidden property key `__go_go_goja_proto_message_pro`
- `go-go-goja/pkg/protogoja/prototype.go:50-73`: `AttachMessagePrototype` stores the token as non-enumerable.
- `go-go-goja/pkg/protogoja/prototype.go:76-91`: `MessagePrototypeFromValue` reads the hidden token.
- `sessionstream/pkg/js/modules/sessionstream/module.go:201-208`: `moduleRuntime.prototypeFromValue` converts a generated namespace token to a fresh prototype message.

Current-state architecture

High-level object flow

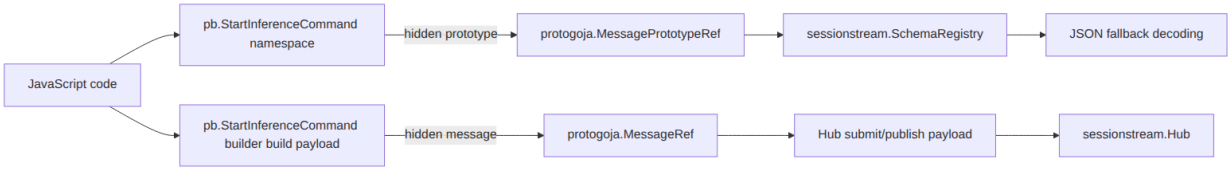

Figure 1: mermaid diagram 1

There are two different hidden-reference paths:

- Schema namespaces use `MessagePrototypeRef`.
- Built message payloads use `MessageRef`.

Conflating the two leads to incorrect fixes. `ss.schemas(...)` needs a schema/prototype token. `hub.submit(...)` needs a payload message.

Current bulk schema path

Current `schemasBuilder` is implemented as:

```
type schemaInput struct {
    Commands map[string]any `json:"commands"`
    Events   map[string]any `json:"events"`
    UIEvents map[string]any `json:"uiEvents"`
    Entities map[string]any `json:"entities"`
}

input := schemaInput{}
if arg := call.Argument(0); !goja.IsUndefined(arg) && !goja.IsNull(arg) {
    if err := m.vm.ExportTo(arg, &input); err != nil { ... }
}
for name, spec := range input.Commands {
    msg := m.mustResolveSchemaSpec(spec)
    registry.RegisterCommand(name, msg)
}
```

Evidence: `pkg/js/modules/sessionstream/api_schemas.go:11-53`.

The resolver for exported specs is:

```
func (m *moduleRuntime) mustResolveSchemaSpec(spec any) proto.Message {
    switch v := spec.(type) {
    case string:
        return resolvePrototype(v)
    case map[string]any:
        if full, ok := v["type"].(string); ok {
            return resolvePrototype(full)
        }
    }
    value := m.vm.ToValue(spec)
    if msg, _, ok := m.prototypeFromValue(value); ok {
        return msg
    }
    panic(TypeError)
}
```

Evidence: `pkg/js/modules/sessionstream/api_schemas.go:56-78`.

This function tries to reconstruct a Goja value after `ExportTo` has already copied the JavaScript value to plain Go data. That is too late for hidden metadata: non-enumerable Go-owned references do not survive as ordinary Go maps.

Current fluent schema path

The fluent methods do not use `ExportTo`:

```
m.mustSet(obj, "registerCommand", func(name string, schema goja.Value) goja.Value {
    msg := m.mustResolveSchemaValue(schema)
    registry.RegisterCommand(name, msg)
```

```
    return obj
})
```

Evidence: pkg/js/modules/sessionstream/api_schemas.go:80-112.

The schema value resolver first checks the hidden prototype token:

```
func (m *moduleRuntime) mustResolveSchemaValue(value goja.Value) proto.Message {
    if msg, _, ok := m.prototypeFromValue(value); ok {
        return msg
    }
    if value != nil && !goja.IsUndefined(value) && !goja.IsNull(value) {
        if full := value.String(); full != "" && full != "[object Object]" {
            return resolvePrototype(full)
        }
    }
    panic(TypeError)
}
```

Evidence: pkg/js/modules/sessionstream/api_schemas.go:114-128.

This is why the existing test passes:

```
const schemas = ss.schemas();
schemas.registerCommand("ChatStartInference", pb.StartInferenceCommand);
```

Evidence: pkg/js/modules/sessionstream/module_test.go:46-61.

Current payload decoding path

Payload handling is separate and mostly correct. `jsValueToProto` first checks for a generated built message:

```
if msg, ok := protogoja.MessageFromValue(value); ok {
    validateMessageType(registry, kind, name, msg)
    return proto.Clone(msg), nil
}
```

Only if that fails does it export the JavaScript value to JSON-compatible data and decode with `protojson` using the registered schema prototype.

Evidence: pkg/js/modules/sessionstream/codec.go:42-68.

This fallback is intentionally lossy because it is a JSON payload boundary. It is not a schema-token boundary. That difference is central to the design.

Reproduction evidence

All scripts used to investigate this ticket are stored in the ticket workspace under:

`ttmp/2026/06/15/SS-GOJA-SCHEMA-MAP-001--analyze-lossy-goja-schema-map-export-and-protobuf-`

Script 01: lossy boundary inventory

`01-search-lossy-js-boundaries.sh` searches `sessionstream` JavaScript-facing code for `ExportTo`, `map[string]any`, `map[string]interface{}`, and `.Export()`.

The relevant output is in `sources/01-lossy-js-boundaries.txt`. The only schema-token ExportTo boundary found is:

```
pkg/js/modules/sessionstream/api_schemas.go:12: Commands map[string]any `json:"commands"`
pkg/js/modules/sessionstream/api_schemas.go:25: if err := m.vm.ExportTo(arg, &input); err
pkg/js/modules/sessionstream/api_schemas.go:64: case map[string]any:
```

Other `map[string]any` usages are JSON output or event/snapshot/fanout shapes, not schema-token inputs.

Script 02: failure reproduction

`02-reproduce-bulk-schema-failure.sh` creates a temporary Go test and removes it after running. It proves both failure modes.

Observed output in `sources/02-reproduce-bulk-schema-failure.txt`:

```
--- FAIL: TestTmpBulkSchemaLowercaseNamespaceFails
Error: GoError: unknown command "ChatStartInference"

--- FAIL: TestTmpBulkSchemaCapitalizedNamespaceFailsAfterExportToMap
Error: TypeError: schema values must be protobuf full-name strings or generated message na
```

Interpretation:

- Lower-case `commands` matches TypeScript but did not populate `schemaInput.Commands` in the reproduction. The registry remained empty.
- Capitalized `Commands` populated the map, but the generated namespace object became a plain map and lost the hidden prototype token.

Script 03: `ExportTo` inspection

`03-inspect-goja-export-loss.sh` runs a small temporary Go program. It directly inspects the exported `schemaInput` values.

Observed output in `sources/03-inspect-goja-export-loss.txt`:

```
CASE 1: lower-case SchemaMap key used by TypeScript contract
ExportTo error: <nil>
commands nil: true

CASE 2: capitalized Go struct field name
ExportTo error: <nil>
commands nil: false
command "ChatStartInference" exported as map[string]interface {}
  keys: [builder clone from is typeName]
  typeName: sessionstream.examples.chatdemo.v1.StartInferenceCommand (string)
  type: <nil> (<nil>)
```

This confirms that the `map[string]any` branch in `mustResolveSchemaSpec` is seeing a plain exported object, not the original generated namespace.

Gap analysis

Gap 1: bulk API does not match TypeScript contract

The TypeScript declaration promises lower-case keys. The implementation uses a Go struct and `ExportTo`, which did not populate from lower-case keys during reproduction.

Impact:

- Users following TypeScript examples can create an empty schema registry without an immediate error.
- The error appears later at `hub.submit`, which makes the root cause hard to discover.

Gap 2: schema tokens cross a lossy conversion boundary

Generated protobuf namespace objects carry hidden non-enumerable Go references. `ExportTo` converts JavaScript objects into public Go values. That is incompatible with schema-token semantics.

Impact:

- Generated namespace values that work in `schemas.registerCommand(...)` fail in `schemas({ ... })`.
- The same public value has inconsistent behavior depending on which API form is used.

Gap 3: fallback object descriptor vocabulary is inconsistent

The bulk `map[string]any` branch recognizes `{ type: "full.name" }`, while generated protobuf namespaces expose `typeName`. The TypeScript `MessageNamespace` contract also names the property `typeName`.

Impact:

- A public object descriptor produced by copying or serializing a generated namespace does not resolve.
- The error message says generated namespace objects are accepted, but the bulk path has already destroyed the generated object identity.

Gap 4: no regression test covers the advertised bulk namespace form

There is an existing test for the fluent path, but not for:

```
ss.schemas({ commands: { ChatStartInference: pb.StartInferenceCommand } })
```

Evidence: `pkg/js/modules/sessionstream/module_test.go:46-61` covers fluent `registerCommand`, not bulk input.

Proposed design

Design goal

Make the following forms equivalent:

```
// Form A: fluent registration
const schemasA = ss.schemas()
  .registerCommand("ChatStartInference", pb.StartInferenceCommand);

// Form B: bulk registration with generated namespace
const schemasB = ss.schemas({
  commands: {
    ChatStartInference: pb.StartInferenceCommand,
  },
});

// Form C: bulk registration with string full names
```

```
const schemasC = ss.schemas({
  commands: {
    ChatStartInference: "sessionstream.examples.chatdemo.v1.StartInferenceCommand",
  },
});

// Form D: bulk registration with explicit object descriptor
const schemasD = ss.schemas({
  commands: {
    ChatStartInference: { typeName: "sessionstream.examples.chatdemo.v1.StartInferenceComm
  },
});
```

Core implementation rule

Do not call ExportTo on schema input. Keep schema leaves as goja.Value until after mustResolveSchemaValue has had the chance to inspect hidden protogoja metadata.

Proposed Go API sketch

Replace schemaInput and mustResolveSchemaSpec(spec any) with a direct Goja object walker:

```
type schemaRegistrar func(name string, msg proto.Message) error

func (m *moduleRuntime) schemasBuilder(call goja.FunctionCall) goja.Value {
    registry := ss.NewSchemaRegistry()
    if m.defaultSchemaRegistry != nil && goja.IsUndefined(call.Argument(0)) {
        return m.wrapSchemaRegistry(m.defaultSchemaRegistry)
    }

    if arg := call.Argument(0); !goja.IsUndefined(arg) && !goja.IsNull(arg) {
        input, ok := arg.(*goja.Object)
        if !ok || input == nil {
            panic(m.vm.NewTypeError("schema input must be an object"))
        }
        m.registerSchemaSection("commands", input.Get("commands"), registry.RegisterComman
        m.registerSchemaSection("events", input.Get("events"), registry.RegisterEvent)
        m.registerSchemaSection("uiEvents", input.Get("uiEvents"), registry.RegisterUIEven
        m.registerSchemaSection("entities", input.Get("entities"), registry.RegisterTimeli
    }

    return m.wrapSchemaRegistry(registry)
}

func (m *moduleRuntime) registerSchemaSection(section string, value goja.Value, register s
    if value == nil || goja.IsUndefined(value) || goja.IsNull(value) {
        return
    }
    obj, ok := value.(*goja.Object)
    if !ok || obj == nil {
        panic(m.vm.NewTypeError("schema %s must be an object", section))
    }
    for _, name := range obj.Keys() {
```

```
        msg := m.mustResolveSchemaValue(obj.Get(name))
        if err := register(name, msg); err != nil {
            panic(m.vm.NewGoError(err))
        }
    }
}
```

Then strengthen `mustResolveSchemaValue` so it supports:

1. generated namespace hidden prototype token;
2. string full names;
3. explicit public object descriptors with `typeName` or `type`.

```
func (m *moduleRuntime) mustResolveSchemaValue(value goja.Value) proto.Message {
    if msg, _, ok := m.prototypeFromValue(value); ok {
        return msg
    }
    if full, ok := schemaFullNameFromValue(value); ok {
        msg, err := m.resolvePrototype(full)
        if err != nil {
            panic(m.vm.NewGoError(err))
        }
        return msg
    }
    panic(m.vm.NewTypeError(
        "schema must be a generated message namespace, protobuf full name, or object with
    ))
}

func schemaFullNameFromValue(value goja.Value) (string, bool) {
    if value == nil || goja.IsUndefined(value) || goja.IsNull(value) {
        return "", false
    }
    if full, ok := value.Export().(string); ok && strings.TrimSpace(full) != "" {
        return full, true
    }
    obj, ok := value.(*goja.Object)
    if !ok || obj == nil {
        return "", false
    }
    for _, key := range []string{"typeName", "type"} {
        prop := obj.Get(key)
        if prop == nil || goja.IsUndefined(prop) || goja.IsNull(prop) {
            continue
        }
        if full, ok := prop.Export().(string); ok && strings.TrimSpace(full) != "" {
            return full, true
        }
    }
    return "", false
}
```

Notes for the intern implementing this:

- Prefer `typeName` before `type` because it is the generated protobuf namespace contract.

- Keep `type` as a compatibility descriptor alias because existing code already recognized it in the exported-map path.
- Do not use `value.String()` for arbitrary objects. It can produce `"[object Object]"` and hides intent. Prefer `value.Export().(string)` for primitive strings and explicit property lookup for objects.
- Continue resolving full names through `resolvePrototype`, which first checks `Options.Prototypes` and then `protoregistry.GlobalTypes`.

Flow after the fix

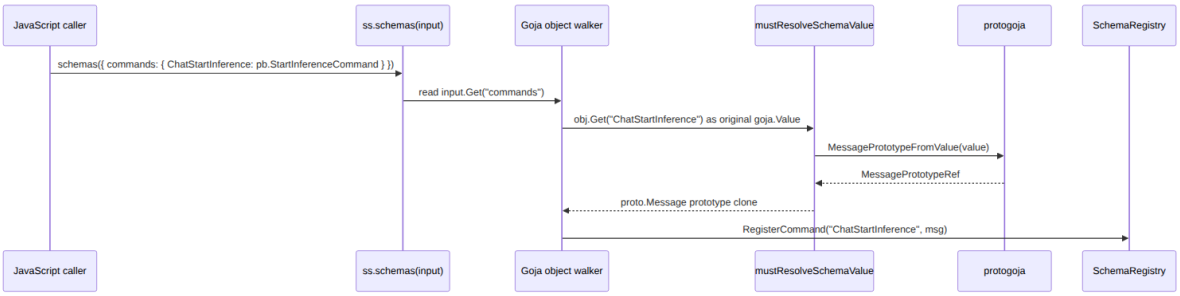

Figure 2: mermaid diagram 2

The important property is that the original `goja.Value` reaches `MessagePrototypeFromValue` unchanged.

Decision records

Decision: Walk Goja schema input directly instead of using `ExportTo`

- **Context:** Generated protobuf namespace objects carry hidden non-enumerable prototype refs. `ExportTo` copies public data and loses the hidden refs.
- **Options considered:**
  1. Keep `ExportTo` and accept `typeName` in `map[string]any`.
  2. Configure a Goja field-name mapper so lower-case section names populate the struct.
  3. Avoid `ExportTo` and walk the original `goja.Value` object.
- **Decision:** Walk the original Goja object directly.
- **Rationale:** This fixes both the lower-case contract mismatch and hidden-token loss. It also makes bulk registration share the same resolver as fluent registration.
- **Consequences:** Slightly more handwritten object traversal code in `api_schemas.go`, but much clearer schema-token semantics.
- **Status:** proposed.

Decision: Keep `typeName` as a fallback, not as the primary namespace path

- **Context:** Generated namespace objects expose public `typeName`, and copied namespace objects retain it. However, public JS properties can be spoofed.
- **Options considered:**
  1. Resolve generated namespaces only through hidden tokens.
  2. Resolve all object descriptors through public `typeName`.
  3. Try hidden tokens first, then public `typeName`/`type` only as descriptor fallback.
- **Decision:** Hidden token first; public `typeName`/`type` fallback second.

- **Rationale:** This preserves the strong generated-object path while accepting documented descriptor forms and copied objects when the type exists in the Go protobuf registry.
- **Consequences:** A malicious script can claim any globally registered protobuf type by string full name, but that was already possible because string full names are an advertised input form.
- **Status:** proposed.

Decision: Do not change `protogoja.MessagePrototypeFromValue` to trust `typeName`

- **Context:** It is tempting to fix this in `go-go-goja` by making `MessagePrototypeFromValue` fall back to public `typeName`.
- **Options considered:**
  1. Make `MessagePrototypeFromValue` trust `typeName`.
  2. Add a separate helper for public type-name extraction.
  3. Leave `go-go-goja` unchanged for this fix.
- **Decision:** Do not change `MessagePrototypeFromValue`; optionally add a separate helper/documentation later.
- **Rationale:** `MessagePrototypeFromValue` currently means "this value carries a Go-owned generated prototype token." Changing it to trust public strings would weaken that semantic boundary and surprise existing consumers.
- **Consequences:** Sessionstream owns the fallback policy because sessionstream intentionally accepts string schema names. `go-go-goja` remains strict and reusable.
- **Status:** proposed.

Are there other lossy `map[string]any` problems?

The inventory script found several `map[string]any` and `.Export()` sites. The conclusion is that `api_schemas.go` is the only confirmed problematic schema-token boundary.

Sites that are intentionally JSON-shaped

These sites build outbound JavaScript objects or event payloads. They do not need to preserve hidden protobuf refs:

- `pkg/js/modules/sessionstream/api_hub.go:143-151`: `snapshotToJS` builds a plain JS snapshot object.
- `pkg/js/modules/sessionstream/api_fanout.go:53-61`: fanout batch is a JSON-style event batch.
- `pkg/js/modules/sessionstream/api_view.go:37`: timeline view lists plain objects for JavaScript consumption.
- `pkg/js/modules/sessionstream/codec.go:140-160`: UI events and timeline entities are converted to plain output maps.

Payload fallback site

`pkg/js/modules/sessionstream/codec.go:59` calls `value.Export()` only after `protogoja.MessageFromValue(value)` fails. That means built generated protobuf payloads take the non-lossy hidden-reference path first. The fallback is for plain JSON-like payload objects.

Caveat: if a JavaScript object contains nested generated `ProtoMessage` values inside an otherwise plain object, the fallback may not preserve them as protobuf messages. That does not appear to be part of the current API contract. If nested protobuf values become a requirement

later, implement a recursive proto-aware conversion helper in `go-go-goja` or `sessionstream` instead of relying on `Export()`.

Hidden sessionstream refs

`pkg/js/modules/sessionstream/module.go:151-168` attaches and reads hidden sessionstream refs for schema registries, hubs, fanouts, and websocket servers. Those refs are consumed from original `goja.Value` arguments. They are not passed through `ExportTo`, so they are not affected by this bug.

Should `go-go-goja` be upgraded?

Recommendation

Do not block the sessionstream fix on `go-go-goja` changes. The sessionstream bug is caused by sessionstream exporting schema-token objects to `map[string]any` before resolving them. A local direct-Goja walker fixes the real boundary.

Useful upstream improvements

A follow-up in `go-go-goja` can still make future consumers safer:

1. **Documentation update:** Add a warning to `pkg/doc/29-protobuf-builders-user-guide.md` and `cmd/protoc-gen-goja-builder/README.md`: generated namespace and message values must be consumed as original `goja.Value`s; `Export`, `ExportTo`, object spread, JSON serialization, or copying only preserves public data such as `typeName`.
2. **Helper for public type names:** Add a separate helper, for example:
   ```
   func PublicTypeNameFromValue(value goja.Value) (protoreflect.FullName, bool)
   ```
   This helper should only read a public `typeName` string. It should not be named like a prototype extractor.
3. **Maybe make generated `typeName` read-only:** `protogoja.ToValue` already uses `defineReadOnly` for built message `typeName` (`ref.go:72`). Generated namespaces currently use `obj.Set("typeName", ...)` in the generator (`generator.go:431`). Making namespace `typeName` read-only would align namespace objects with built messages. It should remain enumerable because it is public API and useful for inspection.
4. **Generator test for export behavior:** Add a test showing that `MessagePrototypeFromValue(namespace)` works on the original namespace, while `ExportTo` produces a plain public object. This documents the boundary rather than hiding it.

Upstream changes to avoid

Avoid changing `protogoja.MessagePrototypeFromValue` so that it constructs a prototype from public `typeName`. That function currently promises hidden-token extraction. Preserving that strong meaning matters for host modules that use it as proof of generated namespace identity.

Implementation guide for a new intern

Phase 0: Read the relevant code

Start with these files in order:

1. `pkg/js/modules/sessionstream/typescript.go`
   - Read the public TypeScript contract for `SchemaMap`, `MessageNamespace`, and `schemas(input)`.
2. `pkg/js/modules/sessionstream/api_schemas.go`
   - Understand the current bulk path and fluent path.
3. `pkg/js/modules/sessionstream/module.go`
   - Read `resolvePrototype` and `prototypeFromValue`.
4. `pkg/js/modules/sessionstream/codec.go`
   - Understand payload decoding so you do not confuse schema tokens with payload messages.
5. `examples/chatdemo/gen/sessionstream/examples/chatdemo/v1/chat_goja.pb.go`
   - Read one generated namespace constructor and one builder `build()` method.
6. `/home/manuel/workspaces/2026-06-12/goja-sessionstream/go-go-goja/pkg/protogoja/prototype`
   - Understand hidden prototype refs.
7. `/home/manuel/workspaces/2026-06-12/goja-sessionstream/go-go-goja/pkg/protogoja/ref.go`
   - Understand hidden built-message refs.

Phase 1: Add failing tests first

Add tests to `pkg/js/modules/sessionstream/module_test.go`.

Recommended tests:

```
func TestSchemasBulkRegisterGeneratedPrototypeNamespaces(t *testing.T) {
    vm := goja.New()
    reg := noderequire.NewRegistry()
    Register(reg, Options{})
    require.NoError(t, chatdemov1.RegisterGojaBuilderFileChatProtoModule(reg, ""))
    reg.Enable(vm)

    _, err := vm.RunString(`
        const ss = require("sessionstream");
        const pb = require("sessionstream.examples.chatdemo.v1");
        const schemas = ss.schemas({
          commands: { ChatStartInference: pb.StartInferenceCommand },
          events: { ChatUserMessageAccepted: pb.UserMessageAcceptedEvent },
          uiEvents: { ChatMessageAccepted: pb.ChatMessageUpdate },
          entities: { ChatMessage: pb.ChatMessageEntity },
        });
        const hub = ss.hub({ schemas });
        hub.command("ChatStartInference", (cmd, session, pub) =>
          pub.publish("ChatUserMessageAccepted", pb.UserMessageAcceptedEvent.builder()
            .messageId("m1")
            .role("user")
            .content(cmd.payload.prompt)
            .build())
        );
        hub.uiProjection((event) => [{
          name: "ChatMessageAccepted",
          payload: pb.ChatMessageUpdate.builder()
```

```
            .messageId(event.payload.messageId)
            .role("user")
            .content(event.payload.content)
            .build(),
        }]);
        hub.timelineProjection((event) => [{
          kind: "ChatMessage",
          id: event.payload.messageId,
          payload: pb.ChatMessageEntity.builder()
            .messageId(event.payload.messageId)
            .role("user")
            .content(event.payload.content)
            .build(),
        }]);
        hub.submit("s-1", "ChatStartInference", pb.StartInferenceCommand.builder().prompt(
    `)
    require.NoError(t, err)
}
```

Also add focused resolver tests:

```
ss.schemas({ commands: { ChatStartInference: "sessionstream.examples.chatdemo.v1.StartInfe
ss.schemas({ commands: { ChatStartInference: { typeName: "sessionstream.examples.chatdemo.
ss.schemas({ commands: { ChatStartInference: { type: "sessionstream.examples.chatdemo.v1.S
```

Make sure each test performs an operation that requires the schema, such as `hub.submit(...)`. Merely constructing `ss.schemas(...)` can hide empty-registry failures.

Phase 2: Refactor `api_schemas.go`

Perform a small targeted refactor:

1. Remove `fmt` if it is only used for `ExportTo` errors.
2. Delete `schemaInput`.
3. Delete `mustResolveSchemaSpec(spec any)`.
4. Add `schemaRegistrar` and `registerSchemaSection`.
5. Change `schemasBuilder` to use `input.Get("commands")`, `input.Get("events")`, `input.Get("uiEvents")`, and `input.Get("entities")`.
6. Update `mustResolveSchemaValue` with explicit `typeName`/`type` object fallback.

Be careful with `goja.Value` checks:

```
if value == nil || goja.IsUndefined(value) || goja.IsNull(value) {
    return
}
```

Do not call `ToObject` on `undefined` or `null` section values.

Phase 3: Validate behavior

Run:

```
go test ./pkg/js/modules/sessionstream -run 'TestSchemas|TestHubCommandProjectionAndSnapsh
go test ./pkg/js/modules/sessionstream -count=1
go test ./... -count=1
```

If the full test suite is too slow or fails due to unrelated integration dependencies, record the exact failure and at least keep the package-level tests passing.

Phase 4: Optional `go-go-goja` follow-up

If asked to improve upstream `go-go-goja`, create a separate ticket/PR. Keep it small:

1. Add docs warning about `ExportTo` lossiness for generated protobuf objects.
2. Consider a helper named clearly around public type names, not hidden prototypes.
3. Add tests under `pkg/protogoja` or `cmd/protoc-gen-goja-builder` documenting current hidden-token behavior.

Do not mix this with the sessionstream patch unless the sessionstream patch truly requires a new upstream API.

Test strategy

Required regression tests

Test	Purpose
bulk generated namespace with lower-case `commands`	Covers the advertised failing API.
bulk string full name with lower-case `commands`	Ensures string form still works after removing `ExportTo`.
bulk `{ typeName }` descriptor	Supports public generated namespace descriptor spelling.
bulk `{ type }` descriptor	Preserves current object descriptor spelling.
fluent `registerCommand` existing test	Guards against regression in the existing working path.
invalid non-object section	Confirms helpful type errors.

Manual smoke snippet

After implementing, this JavaScript should run without error in a Goja test:

```
const ss = require("sessionstream");
const pb = require("sessionstream.examples.chatdemo.v1");

const schemas = ss.schemas({
  commands: {
    ChatStartInference: pb.StartInferenceCommand,
  },
});

const hub = ss.hub({ schemas });
hub.command("ChatStartInference", () => {});
hub.submit(
  "s-1",
  "ChatStartInference",
  pb.StartInferenceCommand.builder().prompt("hello").build()
);
```

Risks and review notes

Risk: accepting public `typeName` expands descriptor forms

This is acceptable because string full names are already allowed. The registry still resolves the name through Go's `Options.Prototypes` or `protoregistry.GlobalTypes`, so arbitrary unknown strings fail.

Risk: iteration order is unspecified

`obj.Keys()` order should not matter because schema registration sections are maps. Duplicate logical names within the same JS object cannot exist as distinct own properties.

Risk: object prototype pollution

Use `obj.Keys()` so only enumerable own keys are registered. Do not walk inherited properties. If stricter behavior is desired later, add tests for `Object.create({ inherited: ... })`.

Risk: default schema registry behavior

Preserve this behavior exactly:

```
if m.defaultSchemaRegistry != nil && goja.IsUndefined(call.Argument(0)) {
    return m.wrapSchemaRegistry(m.defaultSchemaRegistry)
}
```

Passing `{}` should create a new empty registry, not return the default registry.

References

Sessionstream files

- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/pkg/js/modules/sessi` — broken bulk schema decoder and fluent schema registration.
- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/pkg/js/modules/sessi` — module runtime, hidden sessionstream refs, protobuf prototype resolution.
- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/pkg/js/modules/sessi` — payload decoding and JSON fallback.
- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/pkg/js/modules/sessi` — public `SchemaMap` and `MessageNamespace` declarations.
- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/pkg/js/modules/sessi` — existing fluent schema test and integration test patterns.
- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/pkg/sessionstream/sc` — schema registry implementation.
- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/examples/chatdemo/ge` — generated chat demo protobuf Goja namespace and builder code.

Go-go-goja files

- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/go-go-goja/pkg/protogoja/prototype` — hidden prototype token API for generated namespaces.
- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/go-go-goja/pkg/protogoja/ref.go` — hidden built-message reference API for payload values.

- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/go-go-goja/cmd/protoc-gen-goja-builder/internal/generator/generator.go` — generator template for namespace objects and builders.
- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/go-go-goja/pkg/doc/29-protobuf-builders-user-guide.md` — user-facing generated protobuf builder documentation.

Ticket evidence

- `sources/01-lossy-js-boundaries.txt` — inventory of lossy conversion sites.
- `sources/02-reproduce-bulk-schema-failure.txt` — failing temporary tests.
- `sources/03-inspect-goja-export-loss.txt` — direct ExportTo inspection output.
- `scripts/01-search-lossy-js-boundaries.sh` — search script.
- `scripts/02-reproduce-bulk-schema-failure.sh` — failure reproduction script.
- `scripts/03-inspect-goja-export-loss.sh` — export inspection script.

01-investigation-diary

Diary

Goal

This diary records the investigation into lossy Goja schema map handling in `sessionstream`, including the accidental speculative edit that was reverted, the reproduction scripts that were retained in the ticket workspace, and the analysis/design guide produced for the follow-up implementation.

Step 1: Initial schema map investigation and reverted speculative patch

I first inspected the `sessionstream` JavaScript schema registration path to understand the review comment around `map[string]any` and generated protobuf namespaces. The key finding was that `ss.schemas({...})` uses `goja.ExportTo` before resolving schema leaves, while the fluent `schemas.registerCommand(...)` path receives the original `goja.Value` and can still recover the hidden `protogoja` prototype token.

I briefly drafted a direct-Goja walker implementation in `pkg/js/modules/sessionstream/api_schemas.go`, then stopped when the user asked to study the system first and create a ticketed analysis/design guide instead. I reverted the speculative code change with `git checkout -- pkg/js/modules/sessionstream/api_schemas.go`, leaving the repository source unchanged before creating the ticket.

Prompt Context

User prompt (verbatim): "Are there other lossy map strings any to be found? we should do the proper fix."

Assistant interpretation: Search for other lossy `map[string]any` or `ExportTo` boundaries and consider a more principled fix than only adding `typeName` to the existing map branch.

Inferred user intent: Avoid a narrow patch that misses related schema/protobuf object handling problems.

Commit (code): N/A — no code commit; speculative edit was reverted.

What I did

- Read `pkg/js/modules/sessionstream/api_schemas.go`.
- Searched for `ExportTo`, `map[string]any`, and related conversion boundaries.
- Inspected generated protobuf namespace code in `examples/chatdemo/gen/sessionstream/examples/cha`
- Inspected `go-go-goja/pkg/protogoja/prototype.go` and `ref.go` to understand hidden namespace and payload references.
- Drafted and then reverted a direct-Goja walker patch in `api_schemas.go`.

Why

- The review comment suggested that the current map branch loses hidden generated-protobuf prototype metadata.
- A proper fix needed to distinguish schema namespace objects from built protobuf payload objects.

What worked

- The source inspection confirmed that fluent registration already takes the correct `goja.Value` path.
- The initial patch sketch clarified the shape of the likely implementation: walk the original schema input object directly and reuse `mustResolveSchemaValue`.

What didn't work

- I made a speculative edit before creating the requested analysis ticket. That edit was reverted with:

```
git checkout -- pkg/js/modules/sessionstream/api_schemas.go
```

What I learned

- `map[string]any` is not the root concept; the root problem is crossing a lossy JS-to-Go export boundary before consuming hidden `protogoja` metadata.
- `MessagePrototypeFromValue` and `MessageFromValue` are intentionally separate APIs: schema namespaces use prototype refs; built payload messages use message refs.

What was tricky to build

- The tricky part was recognizing that adding `typeName` to the existing map branch is only a compatibility fallback, not the proper fix. Once `ExportTo` has run, the authoritative hidden token is already gone.
- The other tricky point was not confusing JSON payload fallback in `codec.go` with schema-token handling in `api_schemas.go`; payload fallback is intentionally JSON-shaped after checking `protogoja.MessageFromValue`.

What warrants a second pair of eyes

- Confirm whether any consumers intentionally pass capitalized `Commands`/`Events` section names today. The public TypeScript contract uses lower-case names.
- Confirm whether object descriptors with `{ type: ... }` should remain supported when refactoring away from `mustResolveSchemaSpec`.

What should be done in the future

- Implement the direct-Goja schema input walker described in the design doc.
- Add failing tests before changing production code.

Code review instructions

- Start at `pkg/js/modules/sessionstream/api_schemas.go` and compare the bulk `schemas(input)` path with the fluent `registerCommand` path.
- Validate with package-level tests once implementation starts: `go test ./pkg/js/modules/sessionstream -count=1`.

Technical details

- Existing failing area: `pkg/js/modules/sessionstream/api_schemas.go:11-78`.
- Existing working fluent path: `pkg/js/modules/sessionstream/api_schemas.go:80-128`.

- Generated namespace hidden token API: `go-go-goja/pkg/protogoja/prototype.go:50-91`.

Step 2: Ticket creation, reproducible scripts, and intern-ready design guide

After the user asked to pause implementation, I created ticket `SS-GOJA-SCHEMA-MAP-001` and moved the investigation into a docmgr workspace. I also retroactively saved the temporary reproduction and inspection programs as durable scripts under the ticket `scripts/` directory so future readers can rerun the exact evidence-gathering steps.

The main deliverable is a detailed design and implementation guide explaining the session-stream schema registry, Goja generated protobuf namespace objects, hidden `protogoja` references, current failure modes, proposed direct-Goja walker fix, testing strategy, and whether `go-go-goja` itself should change.

Prompt Context

User prompt (verbatim): "hold on, let's study things first and create a new ticket and an analysis + implemengation guide.

We also want to consider if changing ./go-go-goja/ and its protobuf handling itself makes sense to be upgraded?

Create a new ticket and then:

Create a detailed analysis / design / implementation guide that is for a new intern, explaining all the parts of the system needed to understand what it is, with prose paragraphs and bullet point sand pseudocode and diagrams and api references and file references. It should be very clear and technical. Store in the ticket and the nupload to remarkable."

Assistant interpretation: Stop implementation, create a docmgr ticket, investigate evidence first, write a comprehensive intern-ready analysis/design/implementation guide, consider upstream `go-go-goja` changes, store everything in the ticket, and upload the result to reMarkable.

Inferred user intent: Produce a durable design artifact before coding so the eventual fix is evidence-based and reviewable.

Commit (code): N/A — documentation and ticket artifacts only.

What I did

- Created ticket `SS-GOJA-SCHEMA-MAP-001` with docmgr.
- Added design doc `design-doc/01-goja-schema-map-protobuf-namespace-analysis-and-implementation-guide.md`.
- Added diary doc `reference/01-investigation-diary.md`.
- Added ticket tasks for inventory, reproduction, guide writing, validation, and upload.
- Wrote scripts:
  - `scripts/01-search-lossy-js-boundaries.sh`
  - `scripts/02-reproduce-bulk-schema-failure.sh`
  - `scripts/03-inspect-goja-export-loss.sh`
- Captured script outputs under `sources/`.
- Wrote the intern-ready design guide.

Why

- The task changed from implementation to research/design.
- The reproduction scripts preserve exact commands and prevent future investigators from relying on vague memory of temporary experiments.

What worked

- `02-reproduce-bulk-schema-failure.sh` reproduced two failures:
  - lower-case `commands` led to `unknown command "ChatStartInference"`;
  - capitalized `Commands` led to `TypeError: schema values must be protobuf full-name strings or generated message namespace objects`.
- `03-inspect-goja-export-loss.sh` showed that lower-case `commands` did not populate the Go struct in the reproduction, while capitalized `Commands` exported the namespace as `map[string]interface {}` with public keys `[builder clone from is typeName]` and no `type`.

What didn’t work

- The reproduction test intentionally fails under current code. Its output is saved in `sources/02-reproduce-bulk-schema-failure.txt` and should be interpreted as evidence, not as a validation pass.
- reMarkable upload has not yet been completed at the time of this diary step.

What I learned

- The review comment was directionally correct but understated the lower-case key issue: the advertised TypeScript `SchemaMap` uses lower-case keys, and the reproduction showed those keys did not populate `schemaInput` via `ExportTo`.
- The `go-go-goja` hidden-token APIs are behaving as designed; sessionstream should avoid destroying the hidden token before asking `protogoja` to read it.

What was tricky to build

- The reproduction needed to actually submit through the hub. A test that only constructs `ss.schemas({...})` can pass even if the registry is empty, because the error appears later when resolving a command payload.
- The temporary inspection program needed to compare lower-case `commands` with capitalized `Commands` to expose both distinct failure modes.

What warrants a second pair of eyes

- Review the proposed fallback support for both `typeName` and `type` object descriptors.
- Review the recommendation not to change `go-go-goja.MessagePrototypeFromValue` semantics.
- Review whether namespace `typeName` should become read-only in generated `go-go-goja` code as a separate upstream hardening task.

What should be done in the future

- Run `docmgr doctor` after file relations and changelog updates are complete.
- Upload the ticket bundle to reMarkable after validation.
- Implement the direct-Goja walker in a follow-up code change.

Code review instructions

- Review the design doc first, especially “Proposed design”, “Decision records”, and “Implementation guide for a new intern”.
- Rerun evidence scripts from the ticket root if needed:

```
./scripts/01-search-lossy-js-boundaries.sh
./scripts/02-reproduce-bulk-schema-failure.sh
./scripts/03-inspect-goja-export-loss.sh
```

Technical details

- Ticket path: `ttmp/2026/06/15/SS-GOJA-SCHEMA-MAP-001--analyze-lossy-goja-schema-map-export-and-protobuf-namespace-handling`.
- Design doc: `design-doc/01-goja-schema-map-protobuf-namespace-analysis-and-implementation-guide.md`.
- Evidence outputs: `sources/01-lossy-js-boundaries.txt`, `sources/02-reproduce-bulk-schema-failure.txt`, `sources/03-inspect-goja-export-loss.txt`.

Step 3: Retroactively preserve investigation scripts

The user explicitly asked that all scripts I write be saved in the ticket `scripts/` folder, including scripts already used. I confirmed the temporary scripts had been converted into tracked ticket scripts and that their outputs were stored in `sources/`.

Prompt Context

User prompt (verbatim): “save all the scripts you write in the scripts/ folder of the ticket, retroactively too, so we can keep track of your work.”

Assistant interpretation: Ensure every ad-hoc reproduction or inspection script from this investigation exists as a durable script in the ticket workspace.

Inferred user intent: Keep the investigation reproducible and auditable for future continuation or review.

Commit (code): N/A — ticket artifact update only.

What I did

- Saved the lossy boundary search as `scripts/01-search-lossy-js-boundaries.sh`.
- Saved the temporary failing test reproduction as `scripts/02-reproduce-bulk-schema-failure.sh`.
- Saved the temporary `goja.ExportTo` inspection program generator as `scripts/03-inspect-goja-export-loss.sh`.
- Made scripts executable.
- Stored outputs in `sources/`.

Why

- Reproducible scripts make the design guide evidence-backed and easy for an intern to verify.

What worked

- All scripts are now in the ticket workspace and can be rerun from a clean checkout.

What didn’t work

- N/A

What I learned

- The ticket workflow should save scripts before or immediately after first execution; doing so retroactively is possible but easier to miss.

What was tricky to build

- The scripts are stored several directories below the repository root, so they use `git -C "$SCRIPT_DIR" rev-parse --show-toplevel` instead of fragile relative `../../..` paths.

What warrants a second pair of eyes

- Confirm the script set is sufficient, or add a fourth script later for validating the final implementation once code changes begin.

What should be done in the future

- Add `04-validate-implemented-fix.sh` after the actual production patch is written.

Code review instructions

- Start with the scripts in numerical order.
- Compare each script output with the corresponding file under `sources/`.

Technical details

- Scripts are under `ttmp/2026/06/15/SS-GOJA-SCHEMA-MAP-001--analyze-lossy-goja-schema-map-export-and-protobuf-namespace-handling/scripts/`.

tasks

Tasks

TODO

- [ ] Add tasks here
- [x] Inventory lossy Goja conversion boundaries in sessionstream and go-go-goja
- [x] Reproduce the bulk schemas({ ... }) generated namespace failure
- [x] Write intern-ready analysis, design, and implementation guide
- [ ] Validate ticket docs and upload bundle to reMarkable

changelog

Changelog

2026-06-15

- Initial workspace created

2026-06-15

Created evidence-backed analysis and implementation guide for lossy Goja schema map handling; saved reproduction scripts and outputs for future implementation.

Related Files

- /home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/ttmp/2026/06/15/SS-GOJA-SCHEMA-MAP-001–analyze-lossy-goja-schema-map-export-and-protobuf-namespace-handling/design-doc/01-goja-schema-map-protobuf-namespace-analysis-and-implementation-guide.md — Primary deliverable
- /home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/ttmp/2026/06/15/SS-GOJA-SCHEMA-MAP-001–analyze-lossy-goja-schema-map-export-and-protobuf-namespace-handling/reference/01-investigation-diary.md — Chronological investigation record